

Метрики для отслеживания состояния шардов

- Нагрузка по типам операций

```
>_ mongosh: localhost:270... >_ mongosh: shard_1 >_ mongosh: shard_2 +
>_MONGOSH
> const s = db.runCommand({ serverStatus: 1 })
> printjson(s.opcounters)
< {
  insert: Long('521'),
  query: Long('303'),
  update: Long('201'),
  delete: Long('47'),
  getmore: Long('2259'),
  command: Long('9697')
}
shard1 [direct: primary] admin>
```

- Задержки операций (latency)

```
>_ mongosh: localhost:270... >_ mongosh: shard_1 >_ mongosh: shard_2 +
>_MONGOSH
> const s = db.runCommand({ serverStatus: 1 })
> printjson(s.opLatencies)
< {
  commands: {
    latency: Long('2300252'),
    ops: Long('10322'),
    queryableEncryptionLatencyMicros: Long('0')
  },
  reads: {
    latency: Long('734106'),
    ops: Long('2393'),
    queryableEncryptionLatencyMicros: Long('0')
  },
  writes: {
    latency: Long('1002652'),
    ops: Long('504'),
    queryableEncryptionLatencyMicros: Long('0')
  },
  transactions: {
    latency: Long('0'),
    ops: Long('0'),
    queryableEncryptionLatencyMicros: Long('0')
  }
}
shard1 [direct: primary] admin>
```

- Очереди ожидания

```

>_ mongosh: localhost:270... >_ mongosh: shard_1 >_ mongosh: shard_2 +
>_MONGOSH
> const s = db.runCommand({ serverStatus: 1 })
> printjson(s.globalLock?.currentQueue)
< { total: 0, readers: 0, writers: 0 }
shard1 [direct: primary] admin>

```

- CPU
сравнивать user_time_us например между шардами

```

Welcome >_ mongosh: localhost:270... +
>_MONGOSH
> db.adminCommand({ hostInfo: 1 }).system
< {
  currentTime: 2026-03-07T22:49:59.307Z,
  hostname: '1945a6ba7c9d:27030',
  cpuAddrSize: 64,
  memSizeMB: Long('15875'),
  memLimitMB: Long('15875'),
  numCores: 12,
  numCoresAvailableToProcess: 12,
  numPhysicalCores: 6,
  numCpuSockets: 1,
  cpuArch: 'x86_64',
  numaEnabled: false,
  numNumaNodes: 1
}
> db.serverStatus().extra_info
< {
  note: 'fields vary by platform',
  user_time_us: Long('617398'),
  system_time_us: Long('675559'),
  maximum_resident_set_kb: Long('111020'),
  input_blocks: Long('206648'),
  output_blocks: Long('0'),
  page_reclaims: Long('9111'),
  page_faults: Long('1218'),
  voluntary_context_switches: Long('12329'),
  involuntary_context_switches: Long('56'),
  threads: Long('44'),
  pressure: {
    memory: { some: { totalMicros: 0 }, full: { totalMicros: 0 } },
    cpu: { some: { totalMicros: 207261611 }, full: { totalMicros: 0 } },
    io: { some: { totalMicros: 147390684 }, full: { totalMicros: 144631577 } }
  }
}
[mongos] test>

```

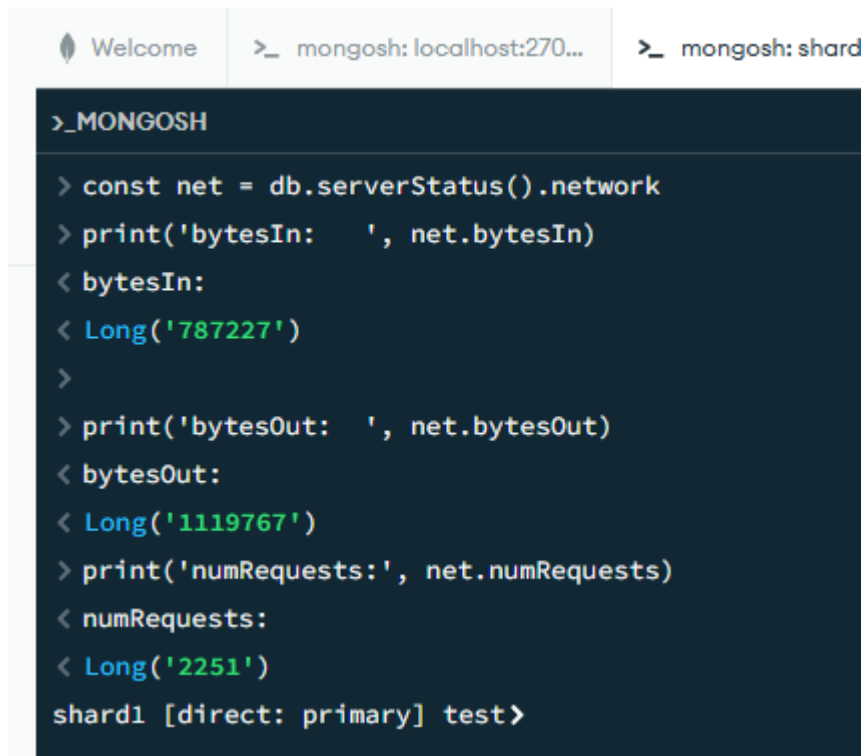
- IO

По bytes read и bytes write можно сравнить нагрузку между шардами

```
Welcome  mongosh: localhost:270...  mongosh: shard_1  +
>_MONGOSH
> const s = db.serverStatus()
> printjson(s.wiredTiger['block-manager'])
< {
  'blocks pre-loaded': 0,
  'blocks read': 126,
  'blocks written': 782,
  'bytes read': 516896,
  'bytes read for internal pages': 0,
  'bytes read for internal pages before decompression and decryption': 0,
  'bytes read for leaf pages': 0,
  'bytes read for leaf pages before decompression and decryption': 0,
  'bytes read via memory map API': 0,
  'bytes read via system call API': 0,
  'bytes written': 5738496,
  'bytes written by compaction': 0,
  'bytes written for checkpoint': 5738496,
  'bytes written for internal pages after compression and encryption': 24616,
  'bytes written for internal pages before compression and encryption': 24616,
  'bytes written for leaf pages after compression and encryption': 6838490,
  'bytes written for leaf pages before compression and encryption': 13851292,
  'bytes written via memory map API': 0,
  'bytes written via system call API': 0,
  'mapped blocks read': 0,
  'mapped bytes read': 0,
  'number of times the file was remapped because it changed size via fallocate or truncate': 0,
  'number of times the region was remapped via write': 0,
  'time spent(usecs) on the most recent linear walk of extents during first-fit allocation': 0
}
```

- Network

Позволяет выявить шард по аномально высокому трафику



The screenshot shows a MongoDB mongosh terminal window with three tabs: 'Welcome', 'mongosh: localhost:270...', and 'mongosh: shard'. The active tab is 'mongosh: shard'. The terminal content is as follows:

```
>_ MONGOSH
> const net = db.serverStatus().network
> print('bytesIn:  ', net.bytesIn)
< bytesIn:
< Long('787227')
>
> print('bytesOut:  ', net.bytesOut)
< bytesOut:
< Long('1119767')
> print('numRequests:', net.numRequests)
< numRequests:
< Long('2251')
shard1 [direct: primary] test>
```

Устранение дисбаланса

Причина дисбаланса заключается в неверном выборе шард-ключа (category), из-за чего все товары категории «Электроника» оказались на одном шарде, что привело в свою очередь к большему кол-ву операций на одном шарде и это можно видеть по opcounters и задержкам. Решение заключается в том что бы разбить категорию на несколько диапазонов для распределения их по разным шардам. Для этого

1. меняем шард-ключ на составной добавляя хешированное поле product_id

```
sh.reshardCollection(  
  "somedb.products",  
  { category: 1, product_id: "hashed" }  
)
```

2. Присваиваем каждому шарду зону через sh.addShardToZone()

```
sh.addShardToZone('shard_1', 'zone_electronics_a')  
sh.addShardToZone('shard_2', 'zone_electronics_b')
```

3. Привязываем зоны к диапазонам через sh.updateZoneKeyRange()

Первая половина product_id (хеш < 0) — на shard_1:

```
sh.updateZoneKeyRange(  
  'somedb.products',  
  { category: 'Electronics', product_id: MinKey },  
  { category: 'Electronics', product_id: NumberLong(0) },  
  'zone_electronics_a'  
)
```

Вторая половина product_id (хеш >= 0) — на shard_2:

```
sh.updateZoneKeyRange(  
  'somedb.products',  
  { category: 'Electronics', product_id: NumberLong(0) },  
  { category: 'Electronics', product_id: MaxKey },  
  'zone_electronics_b'  
)
```

4. Запуск балансировщика и просмотр текущего состояния зон и чанков

```
sh.startBalancer();  
sh.status();
```